



南开大学
Nankai University

《软件测试与维护》实验报告

(2025~2026 学年第二学期)

实验名称：白盒测试实验

学 院：软件学院

姓 名：李宗杭

学 号：2311451

指导老师：张圣林

2026 年 6 月 5 日

目录

1 实验背景	1
2 实验环境	1
3 实验步骤	2
3.1 环境搭建	2
3.2 TodoList 单元测试	2
3.3 WeatherService Mock 测试	8
4 实验结果	13
4.1 TodoList 单元测试结果	13
4.2 WeatherService Mock 测试结果	13
5 实验分析	14
5.1 白盒测试的优势	14
5.2 Mock 技术的作用	14
5.3 代码覆盖率的意义与局限性	15
6 实验总结与心得体会	15
6.1 遇到的问题及解决方法	15
6.2 心得感悟	15

实验 4：白盒测试实验

1 实验背景

白盒测试是一种通过了解程序的内部结构和工作原理来进行测试的方式。在白盒测试中，测试人员需要通过对代码的深入理解，设计测试用例以覆盖代码的各个分支、条件、循环等。白盒测试能够发现隐藏在代码中的逻辑错误、未被测试到的路径以及性能瓶颈等问题，是保证软件内部质量的重要手段。

本次实验将使用 Python 生态中主流的测试工具 `pytest` 进行测试，主要分为两个部分：

1. 对一个待办事项管理程序 (TodoList) 进行单元测试，重点掌握如何设计测试用例覆盖所有代码路径和边界条件
2. 对一个天气服务程序 (WeatherService) 进行集成测试，学习使用 Mock 技术模拟外部依赖，避免实际调用外部 API

通过本次实验，旨在掌握 `pytest` 单元测试框架的使用方法、`pytest-cov` 代码覆盖率分析工具的应用，以及 Mock 技术在隔离外部依赖中的作用，深入理解白盒测试的基本原理和实际应用。

2 实验环境

- 操作系统：Windows 11 家庭版
- Python 版本：3.12.11
- `pytest` 版本：9.0.3
- `pytest-cov` 版本：7.1.0
- `pytest-mock` 版本：3.15.1
- 开发工具：Visual Studio Code
- 终端：Anaconda Powershell Prompt

3 实验步骤

3.1 环境搭建

首先安装本次实验所需的 Python 库：

```
1 pip install pytest pytest-cov pytest-mock requests
```

3.2 TodoList 单元测试

被测代码分析

被测代码包含两个类：

- **Todo 类**：表示单个待办事项，包含**名称**和**完成状态**两个属性，以及**标记完成**和**字符串表示**两个方法
- **TodoList 类**：表示待办事项列表，包含**添加任务**、**删除任务**、**标记任务完成**和**获取所有任务**四个方法

其中需要特别注意的边界条件和分支：

- 添加空名称任务时会抛出 ValueError 异常
- 删除不存在的任务时会抛出 ValueError 异常
- 标记不存在的任务为完成时会抛出 ValueError 异常
- 任务有”Pending”和”Completed”两种状态

测试用例设计

根据代码结构和分支情况，设计以下测试用例：

1. Todo 类测试：

- 测试默认初始化（未完成状态）

- 测试初始化已完成状态
- 测试标记任务为完成
- 测试未完成任务的字符串输出
- 测试已完成任务的字符串输出

2. TodoList 类测试:

- 测试正常添加任务
- 测试添加空名称任务（异常分支）
- 测试删除存在的任务
- 测试删除不存在的任务（异常分支）
- 测试标记存在的任务为完成
- 测试标记不存在的任务（异常分支）
- 测试获取空任务列表
- 测试获取包含任务的列表（包含已完成和未完成状态）

测试代码编写

使用 pytest 编写测试代码，利用 fixture 复用测试对象，提高代码复用性：

```
1  # test_todo_list.py
2  import pytest
3  from todo_list import Todo, TodoList
4
5  # Fixture: 复用测试对象
6  @pytest.fixture
7  def empty_todo_list():
8      """ 返回一个空的待办事项列表 """
9      return TodoList()
10
11  @pytest.fixture
12  def sample_todo_list(empty_todo_list):
```

```
13     """ 返回一个包含 3 个待办事项的列表 """
14     empty_todo_list.add_task(" 学习 Python")
15     empty_todo_list.add_task(" 完成白盒测试实验")
16     empty_todo_list.add_task(" 跑步 30 分钟")
17     return empty_todo_list
18
19 # -----
20 # 测试 Todo 类
21 # -----
22 def test_todo_init_default():
23     """ 测试 Todo 类默认初始化（未完成） """
24     todo = Todo(" 测试任务")
25     assert todo.name == " 测试任务"
26     assert todo.completed is False
27
28 def test_todo_init_completed():
29     """ 测试 Todo 类初始化已完成状态 """
30     todo = Todo(" 已完成任务", completed=True)
31     assert todo.name == " 已完成任务"
32     assert todo.completed is True
33
34 def test_todo_mark_completed():
35     """ 测试标记任务为完成 """
36     todo = Todo(" 待完成任务")
37     todo.mark_completed()
38     assert todo.completed is True
39
40 def test_todo_str_pending():
41     """ 测试未完成任务的字符串输出 """
42     todo = Todo(" 待完成任务")
43     assert str(todo) == " 待完成任务 - Pending"
```

```
44
45 def test_todo_str_completed():
46     """ 测试已完成任务的字符串输出 """
47     todo = Todo(" 已完成任务", completed=True)
48     assert str(todo) == " 已完成任务 - Completed"
49
50     # -----
51     # 测试 TodoList 类
52     # -----
53 def test_add_task_normal(empty_todo_list):
54     """ 测试正常添加任务 """
55     empty_todo_list.add_task(" 新任务")
56     assert len(empty_todo_list.todos) == 1
57     assert empty_todo_list.todos[0].name == " 新任务"
58     assert empty_todo_list.todos[0].completed is False
59
60 def test_add_task_empty_name(empty_todo_list):
61     """ 测试添加空名称任务（抛异常） """
62     with pytest.raises(ValueError, match="Task name cannot be empty"):
63         empty_todo_list.add_task("")
64
65 def test_remove_task_exist(sample_todo_list):
66     """ 测试删除存在的任务 """
67     initial_count = len(sample_todo_list.todos)
68     sample_todo_list.remove_task(" 学习 Python")
69     assert len(sample_todo_list.todos) == initial_count - 1
70     # 验证任务确实被删除
71     task_names = [task.name for task in sample_todo_list.todos]
72     assert " 学习 Python" not in task_names
73
74 def test_remove_task_not_exist(sample_todo_list):
```

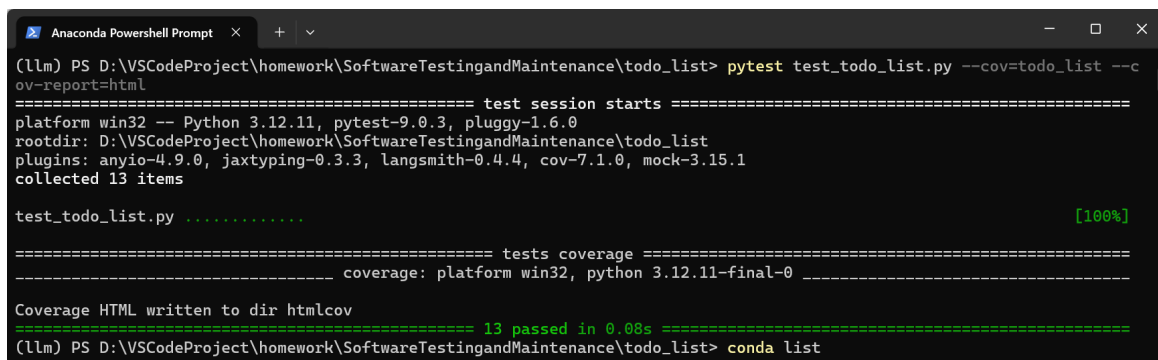
```
75 """ 测试删除不存在的任务（抛异常） """
76 with pytest.raises(ValueError, match="Task with name '不存在的任务' not found"):
77     sample_todo_list.remove_task("不存在的任务")
78
79 def test_mark_task_completed_exist(sample_todo_list):
80     """ 测试标记存在的任务为完成 """
81     sample_todo_list.mark_task_completed("完成白盒测试实验")
82     for task in sample_todo_list.todos:
83         if task.name == "完成白盒测试实验":
84             assert task.completed is True
85             break
86
87 def test_mark_task_completed_not_exist(sample_todo_list):
88     """ 测试标记不存在的任务（抛异常） """
89     with pytest.raises(ValueError, match="Task with name '不存在的任务' not found"):
90         sample_todo_list.mark_task_completed("不存在的任务")
91
92 def test_get_all_tasks_empty(empty_todo_list):
93     """ 测试获取空任务列表 """
94     assert empty_todo_list.get_all_tasks() == []
95
96 def test_get_all_tasks_with_items(sample_todo_list):
97     """ 测试获取包含任务的列表 """
98     sample_todo_list.mark_task_completed("跑步 30 分钟")
99     expected = [
100         "学习 Python - Pending",
101         "完成白盒测试实验 - Pending",
102         "跑步 30 分钟 - Completed",
103     ]
104     assert sample_todo_list.get_all_tasks() == expected
```

测试执行与覆盖率分析

执行以下命令运行测试并生成 HTML 格式的覆盖率报告：

```
1 pytest test_todo_list.py --cov=todo_list --cov-report=html
```

测试执行结果如图 1所示，覆盖率报告如图 2所示。



```
(llm) PS D:\VSCodeProject\homework\SoftwareTestingandMaintenance\todo_list> pytest test_todo_list.py --cov=todo_list --cov-report=html
===== test session starts =====
platform win32 -- Python 3.12.11, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\VSCodeProject\homework\SoftwareTestingandMaintenance\todo_list
plugins: anyio-4.9.0, jaxtyping-0.3.3, langsmith-0.4.4, cov-7.1.0, mock-3.15.1
collected 13 items

test_todo_list.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.12.11-final-0 -----

Coverage HTML written to dir htmlcov
===== 13 passed in 0.08s =====
(llm) PS D:\VSCodeProject\homework\SoftwareTestingandMaintenance\todo_list> conda list
```

图 1: TodoList 测试执行结果

Coverage report: 100%

Files Functions Classes

coverage.py v7.14.1, created at 2026-06-05 01:37 +0800

File ▲	function	statements	missing	excluded	coverage
todo_list.py	Todo.__init__	2	0	0	100%
todo_list.py	Todo.mark_completed	1	0	0	100%
todo_list.py	Todo.__str__	1	0	0	100%
todo_list.py	TodoList.__init__	1	0	0	100%
todo_list.py	TodoList.add_task	3	0	0	100%
todo_list.py	TodoList.remove_task	8	0	0	100%
todo_list.py	TodoList.mark_task_completed	5	0	0	100%
todo_list.py	TodoList.get_all_tasks	1	0	0	100%
todo_list.py	(no function)	10	0	0	100%
Total		32	0	0	100%

coverage.py v7.14.1, created at 2026-06-05 01:37 +0800

图 2: TodoList 代码覆盖率报告

3.3 WeatherService Mock 测试

被测代码分析

被测代码是一个天气服务类 WeatherService，它依赖外部天气 API 获取数据，包含两个主要方法：

- fetch_weather_data：调用外部 API 获取指定城市的天气数据
- get_weather_summary：获取指定城市的天气摘要信息（温度和湿度）

由于该类依赖外部网络服务，直接测试会存在以下问题：

- 测试速度慢，受网络状况影响
- 无法控制 API 返回结果，难以测试边界条件和错误情况
- 可能产生 API 调用费用
- 测试结果不稳定，依赖外部服务的可用性

因此，我们需要使用 Mock 技术模拟外部 API 调用，隔离外部依赖。

测试用例设计

根据代码结构和可能的场景，设计以下测试用例：

1. fetch_weather_data 方法测试：

- 测试成功获取天气数据（状态码 200）
- 测试 API 返回 404 错误（状态码 404）
- 测试 API 返回 500 错误（状态码 500）

2. get_weather_summary 方法测试：

- 测试成功获取天气摘要
- 测试 API 返回缺少 current 字段（格式错误）
- 测试 API 返回缺少 temp_c 字段（格式错误）

测试代码编写

使用 pytest-mock 插件进行 Mock 测试：

```
1  # test_weather_service.py
2  import pytest
3  from weather_service import WeatherService
4
5  # Fixture: 复用 WeatherService 实例
6  @pytest.fixture
7  def weather_service():
8      """ 返回一个使用测试 API 密钥的 WeatherService 实例 """
9      return WeatherService(api_key="test_api_key_123")
10
11  # -----
12  # 测试 fetch_weather_data 方法
13  # -----
14  def test_fetch_weather_data_success(mocked, weather_service):
15      """ 测试成功获取天气数据 """
16      # Mock requests.get 方法
17      mock_get = mocked.patch("weather_service.requests.get")
18      # 创建模拟响应对象
19      mock_response = mocked.MagicMock()
20      mock_response.status_code = 200
21      mock_response.json.return_value = {
22          "location": {"name": "Beijing"},
23          "current": {"temp_c": 25, "humidity": 60},
24      }
25      mock_get.return_value = mock_response
26
27      # 调用被测方法
28      result = weather_service.fetch_weather_data("Beijing")
29
```

```
30 # 验证结果和调用
31 assert result == mock_response.json.return_value
32 # 验证 requests.get 被正确调用了一次, 且 URL 正确
33 expected_url = (
34     "https://api.weatherapi.com/v1/current.json?key=test_api_key_123&q=Beijing"
35 )
36 mock_get.assert_called_once_with(expected_url)
37
38 def test_fetch_weather_data_failure_404(mocked, weather_service):
39     """ 测试 API 返回 404 错误 """
40     mock_get = mocked.patch("weather_service.requests.get")
41     mock_response = mocked.MagicMock()
42     mock_response.status_code = 404
43     mock_get.return_value = mock_response
44
45     # 验证抛出正确的异常
46     with pytest.raises(
47         ConnectionError,
48         match="Failed to fetch weather data for Beijing. Status code: 404",
49     ):
50         weather_service.fetch_weather_data("Beijing")
51
52 def test_fetch_weather_data_failure_500(mocked, weather_service):
53     """ 测试 API 返回 500 错误 """
54     mock_get = mocked.patch("weather_service.requests.get")
55     mock_response = mocked.MagicMock()
56     mock_response.status_code = 500
57     mock_get.return_value = mock_response
58
59     with pytest.raises(
60         ConnectionError,
```

```
61         match="Failed to fetch weather data for Beijing. Status code: 500",
62     ):
63         weather_service.fetch_weather_data("Beijing")
64
65     # -----
66     # 测试 get_weather_summary 方法
67     # -----
68     def test_get_weather_summary_success(mocked, weather_service):
69         """ 测试成功获取天气摘要 """
70         # 直接 Mock fetch_weather_data 方法（更上层，避免重复 Mock requests）
71         mock_fetch = mocked.patch.object(weather_service, "fetch_weather_data")
72         mock_fetch.return_value = {
73             "location": {"name": "Shanghai"},
74             "current": {"temp_c": 28, "humidity": 75},
75         }
76
77         result = weather_service.get_weather_summary("Shanghai")
78
79         assert result == {"city": "Shanghai", "temperature": 28, "humidity": 75}
80         mock_fetch.assert_called_once_with("Shanghai")
81
82     def test_get_weather_summary_invalid_format_missing_current(mocked,
83 ↪ weather_service):
84         """ 测试 API 返回缺少 current 字段（格式错误） """
85         mock_fetch = mocked.patch.object(weather_service, "fetch_weather_data")
86         mock_fetch.return_value = {"location": {"name": "Guangzhou"}} # 缺少 current 字
87 ↪ 段
88
89         with pytest.raises(
90             ValueError, match="Invalid data format received from the weather API."
91         ):
92             weather_service.get_weather_summary("Guangzhou")
```

```

90     weather_service.get_weather_summary("Guangzhou")
91
92 def test_get_weather_summary_invalid_format_missing_temp(mock,
93     ↪ weather_service):
94     """ 测试 API 返回缺少 temp_c 字段（格式错误） """
95     mock_fetch = mocker.patch.object(weather_service, "fetch_weather_data")
96     mock_fetch.return_value = {
97         "location": {"name": "Shenzhen"},
98         "current": {"humidity": 80}, # 缺少 temp_c 字段
99     }
100
101     with pytest.raises(
102         ValueError, match="Invalid data format received from the weather API."
103     ):
104         weather_service.get_weather_summary("Shenzhen")

```

测试执行与覆盖率分析

执行以下命令运行测试并生成 HTML 格式的覆盖率报告：

```
1 pytest test_weather_service.py --cov=weather_service --cov-report=html
```

测试执行结果如图 3所示，覆盖率报告如图 4所示。

```

Anaconda Powershell Prompt
(11m) PS D:\VSCodeProject\homework\SoftwareTestingandMaintenance\weather_service> pytest test_weather_service.py --cov=weather_service --cov-report=html
===== test session starts =====
platform win32 -- Python 3.12.11, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\VSCodeProject\homework\SoftwareTestingandMaintenance\weather_service
plugins: anyio-4.9.0, jaxtyping-0.3.3, langsmith-0.4.4, cov-7.1.0, mock-3.15.1
collected 6 items

test_weather_service.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.12.11-final-0

Coverage HTML written to dir htmlcov
===== 6 passed in 0.09s =====
(11m) PS D:\VSCodeProject\homework\SoftwareTestingandMaintenance\weather_service>

```

图 3: WeatherService 测试执行结果

Coverage report: 100%

Files Functions Classes

coverage.py v7.14.1, created at 2026-06-05 01:38 +0800

File ▲	function	statements	missing	excluded	coverage
weather_service.py	WeatherService.__init__	1	0	0	100%
weather_service.py	WeatherService.fetch_weather_data	5	0	0	100%
weather_service.py	WeatherService.get_weather_summary	7	0	0	100%
weather_service.py	(no function)	5	0	0	100%
Total		18	0	0	100%

coverage.py v7.14.1, created at 2026-06-05 01:38 +0800

图 4: WeatherService 代码覆盖率报告

4 实验结果

4.1 TodoList 单元测试结果

本次 TodoList 单元测试共执行了 13 个测试用例，全部通过，执行时间为 0.08 秒。代码覆盖率达到 100%，所有函数和语句都被覆盖，没有遗漏的代码路径。

从覆盖率报告可以看出：

- Todo 类的所有方法覆盖率均为 100%
- TodoList 类的所有方法覆盖率均为 100%
- 所有异常分支都被测试覆盖
- 所有边界条件都被测试覆盖

4.2 WeatherService Mock 测试结果

本次 WeatherService Mock 测试共执行了 6 个测试用例，全部通过，执行时间为 0.09 秒。代码覆盖率达到 100%，所有函数和语句都被覆盖。

从覆盖率报告可以看出：

- WeatherService 类的所有方法覆盖率均为 100%
- 所有网络错误情况都被模拟和测试

- 所有数据格式错误情况都被模拟和测试
- 没有实际调用外部 API，测试速度快且稳定

5 实验分析

5.1 白盒测试的优势

通过本次实验，我体会到白盒测试的以下优势：

1. **能够发现内部逻辑错误**：白盒测试深入代码内部，能够发现黑盒测试难以发现的逻辑错误，比如条件判断错误、循环边界错误等。
2. **可以精确控制测试覆盖度**：通过代码覆盖率分析工具，可以精确知道哪些代码被执行过，哪些没有，从而有针对性地补充测试用例。
3. **测试用例设计更系统**：白盒测试基于代码结构设计测试用例，能够系统地覆盖所有分支、条件和循环，避免遗漏重要的测试场景。
4. **便于调试和定位问题**：当测试用例失败时，能够快速定位到具体的代码行，大大提高了调试效率。

5.2 Mock 技术的作用

在 WeatherService 测试中，Mock 技术发挥了至关重要的作用：

1. **隔离外部依赖**：通过 Mock 外部 API 调用，完全隔离了测试代码与外部服务的依赖，测试不再受网络状况和外部服务可用性的影响。
2. **提高测试速度**：Mock 调用不需要实际的网络请求，测试仅用 0.09 秒。
3. **便于测试边界条件**：通过 Mock，可以轻松模拟各种难以在真实环境中再现的场景，比如 API 返回 404、500 错误，或者返回格式不正确的数据。
4. **降低测试成本**：避免了实际调用外部 API 产生的费用，同时也不需要搭建复杂的测试环境。

5.3 代码覆盖率的意义与局限性

本次实验中两个测试都达到了 100% 的代码覆盖率，但需要认识到代码覆盖率的意义和局限性：

- **意义**：100% 的代码覆盖率保证了所有代码都被执行过，至少没有明显的语法错误和运行时错误。它是一个重要的质量指标，但不是唯一的指标。
- **局限性**：100% 的代码覆盖率不代表没有 bug。它只能保证代码被执行过，但不能保证代码在所有可能的输入下都能正确运行。例如，一个函数可能在某些输入下会产生错误，但如果测试用例没有覆盖这些输入，即使覆盖率是 100%，bug 仍然存在。

因此，在实际测试中，我们应该追求高代码覆盖率，但同时也要注重测试用例的质量，确保覆盖各种可能的输入和场景。

6 实验总结与心得体会

6.1 遇到的问题及解决方法

1. **如何设计测试用例覆盖所有分支**：在编写 TodoList 测试用例时，一开始不知道如何系统地覆盖所有分支。通过仔细分析代码的控制流图，找出所有的条件判断和异常分支，然后为每个分支设计对应的测试用例，最终实现了 100% 的覆盖率。
2. **如何正确使用 Mock 技术**：在编写 WeatherService 测试用例时，一开始不知道应该 Mock 哪个对象。通过学习 pytest-mock 的文档以及向 AI 寻求帮助，了解到应该 Mock 被测代码中导入的对象，而不是原始模块中的对象。同时，学会了使用 `mock.patch.object` 直接 Mock 类的方法，使测试代码更加简洁。
3. **如何验证异常信息**：在测试异常分支时，需要验证抛出的异常类型和异常信息是否正确。通过使用 pytest 的 `pytest.raises` 上下文管理器，并指定 `match` 参数来匹配异常信息，实现了精确的异常验证。

6.2 心得感悟

通过本次白盒测试实验，我理解了白盒测试的基本原理和实际应用，掌握了 pytest 单元测试框架、pytest-cov 代码覆盖率分析工具和 pytest-mock Mock 测试工具的使用方法。

在 TodoList 单元测试中，我学会了如何基于代码结构设计测试用例，如何覆盖所有的分支和边界条件，以及如何使用 fixture 复用测试对象，提高测试代码的复用性和可维护性。

在 WeatherService Mock 测试中，我体会到了 Mock 技术在隔离外部依赖中的重要作用。它不仅提高了测试速度和稳定性，还让我们能够轻松测试各种难以在真实环境中再现的场景。

本次实验也让我认识到了软件测试的重要性。一个高质量的软件产品不仅需要功能正确，还需要内部代码质量高、可维护性好。白盒测试是保证内部代码质量的重要手段，它能够发现很多黑盒测试难以发现的问题。